

ISutra Developer & Operations Runbook

Engineering Handbook for Setup, Seeding, Testing, Catalog Management, and Deployment

Target Audience: Full-Stack Engineers, DevOps Engineers, QA Specialists, and Open-Source Contributors

🔧 1. Prerequisites & Environment Setup

Required Tools

- **Node.js:** v18.0.0 or higher (tested on Node.js v24.21.0)
- **npm:** v9.0.0 or higher
- **MongoDB:** v5.0+ (Local MongoDB instance or MongoDB Atlas cluster connection string)
- **Git:** For version control

2. Repository Layout

```
ISutra/
├── backend/
│   ├── src/
│   │   ├── config/           # Centralized database & environment config
│   │   ├── controllers/      # Express route handlers
│   │   ├── database/         # Mongoose schemas, seed scripts & verified datasets
│   │   │   ├── verifiedStandards.ts      # 40 Verified BIS standards
│   │   │   ├── verifiedStandardLifecycles.ts # Lifecycle history records
│   │   │   ├── verifiedStandardAmendments.ts # Gazette amendments catalog
│   │   │   ├── verifiedRegulatoryRecords.ts # Mandatory QCO & CRS records
│   │   │   └── seedMongo.ts              # Idempotent database seeder
│   │   ├── middleware/       # Error handler, validation & upload filters
│   │   ├── models/           # Mongoose data models
│   │   ├── routes/           # Express route definitions
│   │   ├── services/         # Core business logic
│   │   │   ├── ai/           # Gemini/OpenAI & offline NLP extraction
│   │   │   ├── standardsMatcher.ts      # Deterministic 6-signal scoring
│   │   │   ├── requirementGapAnalyzer.ts # 4-state reference coverage
│   │   │   ├── standardsRelationshipService.ts # Allied standards graph
│   │   │   ├── standardLifecycleService.ts # 5-stage lifecycle stepper
│   │   │   ├── regulatoryService.ts      # QCO & CRS statutory orders
│   │   │   ├── standardsComparator.ts    # 11-dimension neutral comparator
│   │   │   ├── documentExtractionService.ts # PDF/DOCX magic-byte parser
│   │   │   └── procurementReportService.ts # Audit report generator
│   │   └── index.ts           # Backend Express server entry point
│   ├── test_sih26108_comprehensive.mjs # Master test suite (Tests A-T)
│   ├── test_multilingual.mjs          # Indic script parity tests
│   ├── test_regulatory_certification.mjs # QCO/CRS validation tests
│   ├── test_allied_standards.mjs       # Normative relationship tests
│   ├── test_phaseF.mjs                 # Real document ingestion tests
│   ├── test_differentiation.mjs        # "Why This Standard?" tests
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/      # Reusable UI componentry
│   │   ├── pages/           # React 19 application routes
│   │   ├── services/        # Frontend API client
│   │   ├── App.tsx          # React Router 7 route setup
│   │   └── index.css         # Tailwind CSS v4 styling
│   └── package.json
├── docs/                    # Complete system documentation suite
├── vercel.json              # Serverless cloud deployment configuration
└── README.md                # Project root README
```

3. Configuration & Environment Variables

Copy `.env.example` in both backend and frontend directories:

Backend Configuration (`backend/.env`):

```
# Server Port
PORT=3001
NODE_ENV=development

# MongoDB Connection String (Local or Atlas)
MONGODB_URI=mongodb://127.0.0.1:27017/isutra
# MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.abcd.mongodb.net/isutra?retryWrites=true&w=majority

# AI Provider API Keys (Optional: built-in offline NLP engine operates automatically if keys are absent)
GEMINI_API_KEY=your_gemini_api_key_here
OPENAI_API_KEY=your_openai_api_key_here
DEFAULT_AI_PROVIDER=gemini # Options: 'gemini' | 'openai' | 'offline'

# Feature Flags
ENABLE_OPTIONAL_OCR=false # Set true to enable optional server-side OCR engine
```

NOTE

****Offline Cold-Start Resilience**:** If `MONGODB_URI` is omitted or MongoDB is unreachable, the system automatically falls back to the in-memory verified reference catalog. If external AI API keys are omitted, the system falls back to the built-in offline deterministic NLP extractor. The application never crashes due to missing external cloud credentials.

4. Local Installation & Startup

Step 1: Install Dependencies

```
# Install backend dependencies
cd backend
npm install

# Install frontend dependencies
cd ../frontend
npm install
```

Step 2: Seed the Database

Seed the 40 verified Indian Standards into MongoDB:

```
cd backend
npm run seed:standards
```

(The seeder is **strictly idempotent**: running it multiple times preserves exactly 40 verified records with 0 duplicates).

Step 3: Start the Backend Dev Server

```
cd backend
npm run dev
```

Backend runs at `http://localhost:3001` with API routes under `/api`.

Step 4: Start the Frontend Dev Server

In a separate terminal:

```
cd frontend
npm run dev
```

Frontend development server starts at `http://localhost:5173`. Open in your browser.

5. Running the Automated Test Suites

ISutra includes six automated test suites containing **over 280 passing test assertions** with **0 failures**:

```
# 1. Comprehensive SIH26108 Capability Test Suite (Tests A through T)
node backend/test_sih26108_comprehensive.mjs

# 2. Multilingual Procurement Input Verification Suite (English, Hindi, Telugu)
node backend/test_multilingual.mjs

# 3. Regulatory & Certification Intelligence Test Suite (QCO, CRS, Hallmarking)
node backend/test_regulatory_certification.mjs

# 4. Allied Standards Intelligence Test Suite (Normative, Test Methods, Safety)
node backend/test_allied_standards.mjs

# 5. Real Document Ingestion & Report Test Suite (PDF, DOCX, Scanned Docs)
node backend/test_phaseF.mjs

# 6. Explainable Decision Trail Test Suite ("Why This Standard?" & Scoring Weights)
node backend/test_differentiation.mjs
```

Test Coverage Highlights:

- **Scoring Parity:** Verifies that identical specifications in English, Hindi, and Telugu produce identical deterministic relevance scores (80% score parity).
- **Contradiction Filtering:** Asserts that underground mining luminaires receive an explicit contradiction penalty when matching highway street light tenders.
- **Traceability Chain:** Verifies that every recommendation returns all 5 stages of the custody chain.
- **Document Ingestion:** Asserts that real PDFs and DOCX files parse successfully and scanned PDFs are safely rejected with `422`.

6. Standard Ingestion Protocol

To add new Indian Standards to the catalog while preserving zero-hallucination integrity, use the schema validator (`standardsIngestionService.ts`).

Ingestion Validation Endpoint:

POST `/api/standards/validate-ingestion`

```
{
  "id": "is-4984",
  "standard_number": "IS 4984: 2016",
  "title": "High Density Polyethylene Pipes for Water Supply - Specification",
  "category": "Piping & Fluid Distribution",
  "subcategory": "HDPE Pipes",
  "scope": "This standard specifies requirements for high density polyethylene (HDPE) pipes intended for th",
  "source_url": "https://services.bis.gov.in/php/BIS_2.0/bisconnect/nowyourstandards/indian_standards/isde",
  "last_verified": "2026-03-01T00:00:00.000Z",
  "status": "active"
}
```

Required Checklist Before Adding a Standard:

1. Verify the standard's active status on `services.bis.gov.in` .
2. Confirm the `source_url` uses HTTPS on an authoritative `.gov.in` or `.crsbis.in` domain.
3. Check for active Quality Control Orders (QCOs) on `egazette.gov.in` .
4. Ensure the scope text is quoted verbatim from official BIS publications.

7. Production Deployment (Vercel & Docker)

Cloud Deployment on Vercel

The repository includes a root `vercel.json` configured for serverless full-stack execution:

1. Connect your GitHub repository to Vercel.
2. Configure Environment Variables in the Vercel Project Settings:
 - `MONGODB_URI` : Your MongoDB Atlas connection URI.
 - `GEMINI_API_KEY` or `OPENAI_API_KEY` : Your AI API key.
 - `NODE_ENV` : `production` .
3. Trigger deployment. Vercel automatically builds both the frontend SPA and backend serverless endpoints.

Containerization (Docker)

For self-hosted government data center deployments:

```
# Multi-stage production Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN cd backend && npm install && npm run build
RUN cd frontend && npm install && npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/backend/dist ./backend/dist
COPY --from=builder /app/backend/node_modules ./backend/node_modules
COPY --from=builder /app/backend/package.json ./backend/package.json
COPY --from=builder /app/frontend/dist ./frontend/dist

EXPOSE 3001
CMD ["node", "backend/dist/index.js"]
```

8. Troubleshooting & FAQ

Q: Why does the system return 422 for a PDF file?

A: The uploaded PDF is a scanned bitmap without an embedded text layer. In strict mode, the system protects against OCR hallucination by rejecting scanned documents and guiding the user to upload a digital export.

Q: What happens if MongoDB Atlas experiences network latency?

A: The connection manager (`backend/src/config/database.ts`) catches timeouts gracefully and switches into memory-cached verified dataset mode. The user experiences zero request failures.

Q: How do we update scoring factor weights?

A: Modify `DEFAULT_MATCHING_WEIGHTS` in `standardsMatcher.ts` . Ensure the sum of all six weights equals exactly \$1.00\$ (\$100%\$). Run `node backend/test_differentiation.mjs` to verify weight calibration.